Gerenciamento de processos seguros

Pedro Boeira Webber

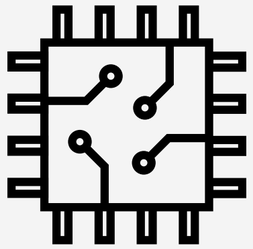
Tópicos

- Contexto histórico
- Kernel vs user space
- syscalls
- Demonstração

Contexto histórico

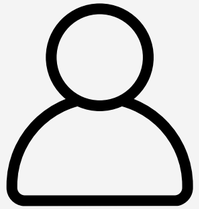
1960s–’70s: Mainframes & Multics	• Multics rings 0–7 • UNIX ring 0/3
1980s–’90s: x86	• Intel introduz modelo x86 • Windows NT kernel/user
2000s–Atual: Virtualização	• Containers (namespaces, cgroups) • eBPF/seccomp para filtros de syscall

Kernel space vs user space



Kernel Space

- Ring 0 / modo privilegiado
- Acesso ao hardware
- Drivers, scheduler, gerenciador de memória



User space

- Ring 3 / user mode
- Isolado por processo
- Precisa usar syscalls para operações privilegiadas

syscalls

Entrada no kernel

- **Legado**: int 0x80 (x86-32)
- **Moderno**: syscall / sysenter (x86-64)
- **ARM:** trap svc #0

Dispatch

- Tabela de syscalls → handler (ex: sys_read)

User-Space Wrapper

- glibc read()

syscalls

```c
#include <unistd.h>
#include <sys/syscall.h>

int main() {
    const char msg[] = "Hello, world\n";
    syscall(SYS_write, 1, msg, sizeof(msg) - 1);
    return 0;
}
```

Imprime no stdout

```c
#include <sys/syscall.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = syscall(SYS_fork);

    if (pid == 0) {
        // Child
        const char msg[] = "Child process\n";
        syscall(SYS_write, 1, msg, sizeof(msg)-1);
    } else {
        // Parent
        const char msg[] = "Parent process\n";
        syscall(SYS_write, 1, msg, sizeof(msg)-1);
    }

    return 0;
}
```

Fork

Demonstração

- demo_exec.c
- demo_seccomp.c
- gcc demo_seccomp.c -lseccomp -o demo_seccomp
- dmesg | tail

- type=1326 # Evento do Seccomp
- pid=652394
- comm="demo_seccomp" # Comando que trigouo evento
- exe="..." # caminho
- sig=31 # Signal 31 = SIGSYS → syscall violation
- arch=c000003e # arquitetura x86_64
- syscall=59 # Syscall number 59 = execve